

68M 与 TinyLlama-1.1B 草稿模型推测解码对比

实验目的

上一轮 68M 草稿模型接受率偏低，因此本轮将草稿模型替换为 TinyLlama/TinyLlama_v1.1，仍与 Llama-2-7b-chat-hf target 在同一张 NVIDIA RTX PRO 6000 上运行，通信时延置零，比较固定起草长度和熵感知自适应起草长度的效果。

TinyLlama 原本本地目录缺少权重。本轮已先将权重下载到：

```
experiments/Model/TinyLlama-1.1B-Draft/pytorch_model.bin
```

下载后目录大小约 4.2GB，词表大小为 32000，BOS/EOS 与 7B target 一致。

实验设置

项目	设置
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition
Target	experiments/Model/Llama-7B-Chat-Target
Draft A	experiments/Model/Llama-68M-Draft
Draft B	experiments/Model/TinyLlama-1.1B-Draft
数据集	gsm8k, mbpp, wikitext-103-raw-v1
样本数	每个数据集 3 条
Decode 长度	每条约 16 个输出 token
固定长度	$g \in \{2, 4, 8\}$
自适应上限	$g_{plan}=8$
通信时延	忽略

TinyLlama 运行命令：

```
OMP_NUM_THREADS=1 /root/miniconda3/envs/SD_Blackwell/bin/python \
  experiments/speculative_latency_experiment.py \
  --draft-model experiments/Model/TinyLlama-1.1B-Draft \
  --datasets gsm8k mbpp wikitext \
  --samples-per-dataset 3 \
```

```
--max-new-tokens 16 \  
--fixed-draft-lengths 2 4 8 \  
--adaptive-plan-g 8 \  
--profile-repeat 2 \  
--output-dir experiments/speculative_latency_results_tinyllama
```

TinyLlama 结果目录：

```
experiments/speculative_latency_results_tinyllama/  
run_20260705_163810/
```

关键公式

实验仍使用观测 TPOT：

$$J \sim \text{Tround}(g)K + 1,$$

其中 K 是本轮实际 accepted draft token 数。相对 target-only 的加速比为：

$$\text{speedup} = J/J \sim.$$

speedup > 1 表示推测解码快于 target-only；本轮所有均值仍低于 1。

跨数据集平均结果

Draft	策略	TPOT ms	speedup	latency reduction	接受率	平均上传 g	平均生成 g
68M	target-only	13.67	-	-	-	-	-
68M	fixed g=2	22.36	0.65	-64.2%	37.6%	2.00	2.00
68M	fixed g=4	21.50	0.71	-57.9%	38.1%	4.00	4.00
68M	fixed g=8	23.83	0.66	-75.2%	38.7%	8.00	8.00
68M	adaptive	23.00	0.62	-68.8%	36.4%	1.50	2.49
TinyLlama-1.1B	target-only	13.64	-	-	-	-	-
TinyLlama-1.1B	fixed g=2	23.54	0.59	-72.9%	68.4%	2.00	2.00
TinyLlama-1.1B	fixed g=4	25.09	0.56	-84.3%	66.4%	4.00	4.00
TinyLlama-1.1B	fixed g=8	31.99	0.46	-135.1%	67.4%	8.00	8.00

Draft	策略	TPOT ms	speedup	latency reduction	接受率	平均上传 g	平均生成 g
TinyLlama-1.1B	adaptive	29.37	0.47	-115.5%	64.6%	1.34	2.32

各数据集最佳固定长度

Draft	数据集	最佳固定策略	TPOT ms	speedup	接受率
68M	gsm8k	fixed g=4	17.92	0.81	49.8%
68M	mbpp	fixed g=2	28.47	0.47	10.2%
68M	wikitext	fixed g=4	17.54	0.85	51.4%
TinyLlama-1.1B	gsm8k	fixed g=2	25.87	0.53	61.5%
TinyLlama-1.1B	mbpp	fixed g=2	24.11	0.58	63.9%
TinyLlama-1.1B	wikitext	fixed g=2	20.65	0.68	79.8%

结论

TinyLlama-1.1B 显著提高了接受率。跨数据集平均接受率从 68M 的约 38% 提升到 TinyLlama 的约 66%-68%；其中 mbpp 最明显，最佳固定策略接受率从 10.2% 提升到 63.9%。

但 TinyLlama 并没有带来端到端加速。原因是同卡实验中 draft token 生成也计入时延，TinyLlama 的 profiling 草稿生成成本约为 13.96 ms/token，而 68M 约为 7.17 ms/token。接受率提升不足以抵消更大的 draft 生成和 cache 维护开销。

固定长度方面，TinyLlama 的最佳固定策略在三个数据集上都退化到 g=2，说明更长草稿虽然接受率仍高，但额外生成成本过大。自适应策略能减少上传长度，平均上传 g=1.34，但平均生成 g=2.32，仍慢于固定 g=2。

因此，在“两个模型同卡、通信时延为零”的设置下，TinyLlama-1.1B 更像是提高接受率的验证点，而不是最优时延点。若后续要让 TinyLlama 体现优势，应考虑分布式部署、通信/边侧负载建模、更长输出长度、批量 target 验证，或使用更高效的 draft KV/cache 更新实现。

产物

- TinyLlama 权重: experiments/Model/TinyLlama-1.1B-Draft/pytorch_model.bin
- TinyLlama 原始结果: experiments/speculative_latency_results_tinyllama/run_20260705_163810/raw_results.csv

- TinyLlama 汇总结果: experiments/speculative_latency_results_tinyllama/run_20260705_163810/summary_by_dataset_strategy.csv
- TinyLlama 元数据: experiments/speculative_latency_results_tinyllama/run_20260705_163810/metadata.json